



Instituto Politécnico Nacional
Escuela Superior de Cómputo
Ingeniería en Sistemas Computacionales

Compiladores

Proyecto Final: Garabato (Logo)
Manual Usuario

Profesor:
Tecla Parra Roberto

Equipo:
Luis Ernesto Mérida de León
Héctor Said Ferreira Rodríguez
Diego García Cuanalo
Francisco Omar Gutiérrez Silva

Grupo: 5CM2

16 / 06 / 2026

Introducción

Logo es un lenguaje de programación educativo creado en 1967 por Seymour Papert y Wallace Feurzeig, basado en el lenguaje Lisp. Su objetivo desde el inicio fue didáctico: permitir que cualquier persona, sin importar su nivel de experiencia, pudiera aprender a programar de forma visual e intuitiva.

El elemento central de Logo es la tortuga: un cursor gráfico que recibe instrucciones de movimiento y con ellas traza figuras en pantalla. En sus orígenes era un robot físico; con el tiempo se volvió una representación virtual sobre una pantalla, que es lo que implementa Garabato.

Garabato es un intérprete del lenguaje Logo construido en Java como proyecto final de la materia de Compiladores. El usuario escribe instrucciones Logo en una caja de texto y la tortuga virtual las ejecuta en tiempo real, dibujando figuras directamente en el lienzo de la aplicación. El nombre hace alusión al acto de dibujar libremente, que es exactamente lo que el programa permite hacer.

Objetivo

Este manual tiene como propósito guiar al usuario final en el uso de Garabato: cómo iniciar el programa, qué hace cada elemento de la interfaz, qué comandos se pueden usar y cómo combinarlos para crear figuras de distinta complejidad, desde un simple cuadrado hasta fractales recursivos.

Requerimientos

Para poder ejecutar Garabato solo se necesita tener instalado en el equipo:

- Java JDK 11 o superior (incluye todo lo necesario para correr el archivo .jar).

No se requiere ningún IDE, servidor ni configuración adicional. El programa se distribuye como un ejecutable autocontenido listo para usarse.

Cómo Ejecutar el Programa

Garabato se distribuye como un archivo JAR ejecutable. Dependiendo del sistema operativo, hay dos formas de abrirlo:

Opción A — Doble clic (Windows / macOS)

En la mayoría de los equipos con Java instalado, basta con hacer doble clic sobre el archivo Ejecutable_Garabato.jar y la ventana del programa se abrirá directamente.

Opción B — Desde la terminal

Abre una terminal (cmd en Windows, Terminal en macOS/Linux), navega a la carpeta donde está el archivo y escribe:

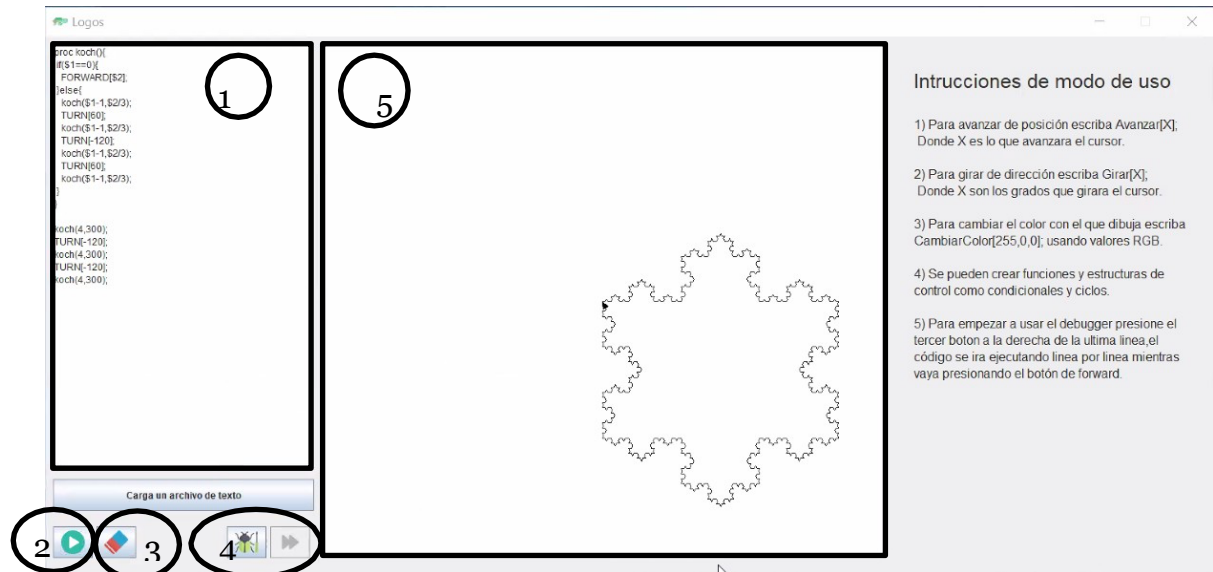
```
java -jar Ejecutable_Garabato.jar
```

También se incluyen scripts de acceso rápido: Ejecutar-Garabato.bat para Windows y Ejecutar-Garabato.sh para macOS/Linux, que hacen exactamente lo mismo con un doble clic.

Tip: si al hacer doble clic en el .jar no pasa nada, verifica que Java esté correctamente instalado ejecutando "java -version" en la terminal. Debe mostrar una versión 11 o superior.

Descripción de la Interfaz Gráfica

Al ejecutar el programa aparece una ventana única dividida en tres zonas principales: el panel izquierdo con el área de código, el lienzo central donde se dibuja, y un panel de instrucciones rápidas a la derecha.



Zona	Elemento	Descripción
1	Área de código	Caja de texto multilínea donde se escribe el programa Logo. Acepta cualquier combinación de instrucciones, estructuras de control, funciones y procedimientos.
2	Botón Run	Compila y ejecuta todo el código escrito en el área de texto de una sola vez. El resultado aparece dibujado en el lienzo al instante.
3	Botón Clear	Borra el contenido del área de texto y reinicia el lienzo y el estado de la tortuga por completo. Útsalo para empezar desde cero.
4	Botón Debug	Activa el modo de ejecución paso a paso. Al presionarlo, el código se compila pero no se ejecuta todavía. La tortuga espera. El área de texto y el botón Run se desactivan mientras este modo está activo. Para salir del modo Debug, vuelve a presionar el mismo botón.
5	Botón Siguiente	Solo disponible en modo Debug. Cada clic ejecuta una instrucción del programa y actualiza el lienzo. Permite observar el dibujo construirse línea por línea.
6	Lienzo / Panel de dibujo	Zona blanca central donde la tortuga traza las figuras. El origen (0,0) está en el centro del lienzo. Un pequeño triángulo negro indica la posición y dirección actual de la tortuga en todo momento.
7	Carga archivo de texto	Botón que abre un explorador de archivos para cargar un archivo .txt con código Logo previamente guardado. El contenido se carga directamente en el área de código.
8	Panel de instrucciones	Zona de texto fija en el lado derecho con un recordatorio rápido de los comandos básicos del lenguaje.

La tortuga siempre arranca en el centro del lienzo (posición 0,0) apuntando hacia la derecha. El triángulo negro te indica en todo momento hacia dónde se moverá la siguiente instrucción FORWARD.

Comandos del Lenguaje

Todas las instrucciones de Garabato distinguen mayúsculas de minúsculas y deben terminar con punto y coma (;). A continuación se describen todos los comandos disponibles.

Movimiento y Dibujo

FORWARD[n]

Desplaza la tortuga n píxeles hacia adelante en la dirección en que apunta. Si el lápiz está abajo (PenDOWN), traza una línea en ese recorrido. El valor de n puede ser negativo para retroceder.

```
FORWARD[100];    // avanza 100 píxeles
FORWARD[-50];    // retrocede 50 píxeles
```

TURN[n]

Gira la orientación de la tortuga n grados en sentido horario. Los valores negativos giran en sentido antihorario.

```
TURN[90];        // gira 90 grados a la derecha
TURN[-45];       // gira 45 grados a la izquierda
```

COLOR[R,G,B]

Cambia el color del trazo. Los valores R, G y B son números entre 0 y 255 que corresponden a los canales rojo, verde y azul.

```
COLOR[255,0,0];  // rojo
COLOR[0,255,0];  // verde
COLOR[0,0,255];  // azul
COLOR[255,165,0]; // naranja
COLOR[128,0,128]; // morado
```

Control del Lápiz

PenUP[]

Levanta el lápiz. La tortuga se moverá por el lienzo sin dejar rastro. Úsalo para reposicionarte sin dibujar.

PenDOWN[]

Baja el lápiz. A partir de este punto, cada FORWARD dejará una línea en el lienzo. El lápiz inicia en modo DOWN por defecto al arrancar.

```
PenUP[];
FORWARD[150];    // se mueve 150 píxeles sin dibujar
PenDOWN[];
FORWARD[100];    // ahora sí dibuja
```

Variables

Puedes crear variables asignándoles un valor. Las variables almacenan números y se pueden usar en cualquier expresión.

```
lado = 100;
FORWARD[lado];
TURN[90];
```

```
FORWARD[lado * 2];
```

Estructuras de Control

Ciclo for

Repite un bloque de instrucciones un número determinado de veces. La variable de control se incrementa en cada iteración.

```
for(i = 0; i < 4; i = i + 1) {  
    FORWARD[100];  
    TURN[90];  
}
```

La condición del for usa < (menor estricto), no <=. Para repetir 4 veces escribe $i < 4$, no $i \leq 4$.

Ciclo while

Repite un bloque mientras la condición sea verdadera.

```
x = 0;  
while(x < 5) {  
    FORWARD[x * 20];  
    TURN[72];  
    x = x + 1;  
}
```

Condicional if / else

Ejecuta un bloque u otro dependiendo de si la condición es verdadera o falsa.

```
nivel = 3;  
if(nivel > 0) {  
    FORWARD[100];  
} else {  
    TURN[180];  
}
```

Operadores disponibles

Tipo	Operadores	Ejemplo
Aritméticos	+ - * /	lado * 2 + 10
Comparación	== != < <= > >=	x < 10
Lógicos	&& !	x > 0 && y < 5
Asignación	=	lado = 100

Procedimientos

Los procedimientos son bloques de código reutilizables que no devuelven ningún valor. Se declaran con la palabra clave proc y se llaman por su nombre.

```
proc cuadrado() {  
    for(i = 0; i < 4; i = i + 1) {  
        FORWARD[$1];  
        TURN[90];  
    }  
}  
  
cuadrado(100);    // dibuja un cuadrado de lado 100  
cuadrado(50);     // dibuja uno más chico
```

Los parámetros se acceden dentro del cuerpo con \$1, \$2, \$3... dependiendo del orden en que se pasan al llamar el procedimiento.

Funciones

Las funciones son iguales a los procedimientos, pero devuelven un valor usando return. Se declaran con func.

```
func doble() {  
    return $1 * 2;  
}  
  
FORWARD[doble(50)]; // avanza 100 píxeles
```

Cargar código desde un archivo

Si tienes un programa Logo guardado en un archivo de texto (.txt), puedes cargarlo directamente sin tener que escribirlo de nuevo. Haz clic en el botón “Carga un archivo de texto”, selecciona el archivo en el explorador y su contenido aparecerá automáticamente en el área de código, listo para ejecutarse.

El archivo debe ser texto plano (.txt). Los archivos con extensión diferente no serán visibles en el explorador de archivos del programa.

Modo Debug (Paso a Paso)

El modo Debug es útil cuando quieres entender cómo se construye una figura o cuando quieres identificar dónde algo no está saliendo como esperas. Permite ejecutar el programa una instrucción a la vez y observar el resultado en el lienzo después de cada paso.

Cómo usarlo

1. Escribe tu código Logo en el área de texto normalmente.
2. Presiona el botón Debug (el ícono de bicho/insecto). El código se compila y el programa entra en modo paso a paso. El área de texto y el botón Run se desactivan.
3. Presiona el botón Siguiente (la flecha) una vez por cada instrucción que quieras ejecutar. El lienzo se actualiza después de cada clic.
4. Cuando termines, presiona el botón Debug de nuevo para salir del modo paso a paso. El área de código y el botón Run vuelven a activarse.

Mientras el modo Debug esté activo no puedes editar el código. Si quieres modificar algo, sal del modo primero y luego vuelve a activarlo después de hacer cambios.

Ejemplos de Programas

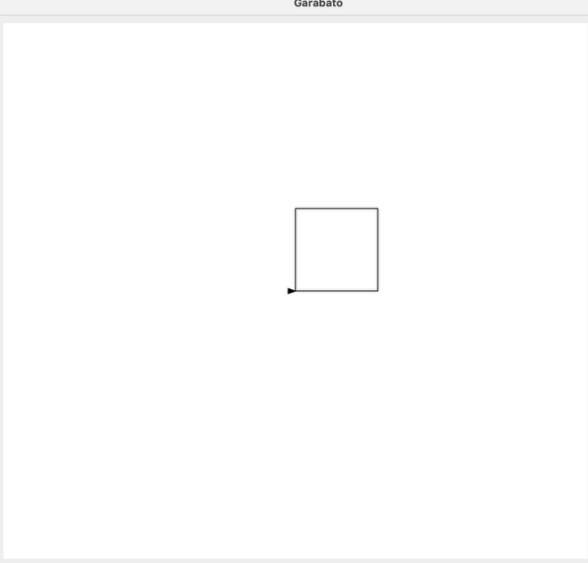
A continuación se presentan ejemplos listos para copiar y ejecutar en Garabato, ordenados de menor a mayor complejidad.

Cuadrado básico

```
for(i = 0; i < 4; i = i + 1) {  
  FORWARD[100];  
  TURN[90];  
}
```

Carga un archivo de texto





Intrucciones de modo de uso

- 1) Para avanzar de posición escriba FORWARD[X];
Donde X es lo que avanzara el cursor.
- 2) Para girar de dirección escriba TURN[X];
Donde X son los grados que girara el cursor.
- 3) Para cambiar el color con el que dibuja escriba COLOR[255,0,0]; usando valores RGB.
- 4) Se pueden crear funciones y estructuras de control como condicionales y ciclos.
- 5) Para empezar a usar el debugger presione el tercer boton a la derecha de la ultima linea, el código se ira ejecutando linea por linea mientras vaya presionando el botón de forward.

El ejemplo más sencillo: avanzar y girar cuatro veces.

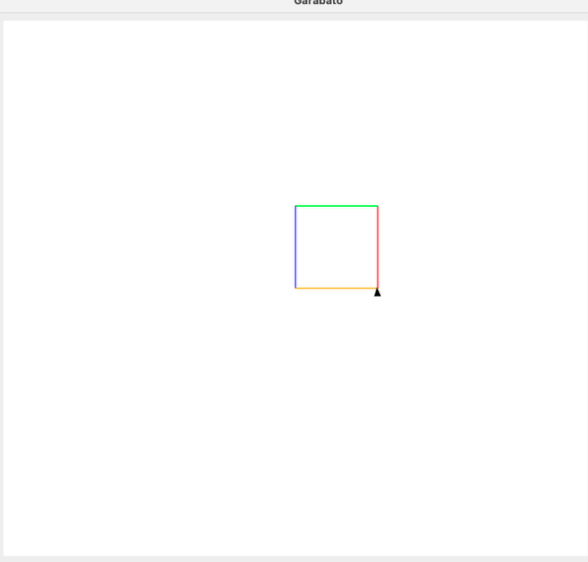
```
for(i = 0; i < 4; i = i + 1) {  
  FORWARD[100];  
  TURN[90];  
}
```

Cuadrado con colores

```
PenUP[];  
FORWARD[100];  
TURN[90];  
PenDOWN[];  
COLOR[255,0,0]; FORWARD[100]; TURN[90];  
COLOR[0,255,0]; FORWARD[100]; TURN[90];  
COLOR[0,0,255]; FORWARD[100]; TURN[90];  
COLOR[255,165,0]; FORWARD[100]; TURN[90];
```

Carga un archivo de texto





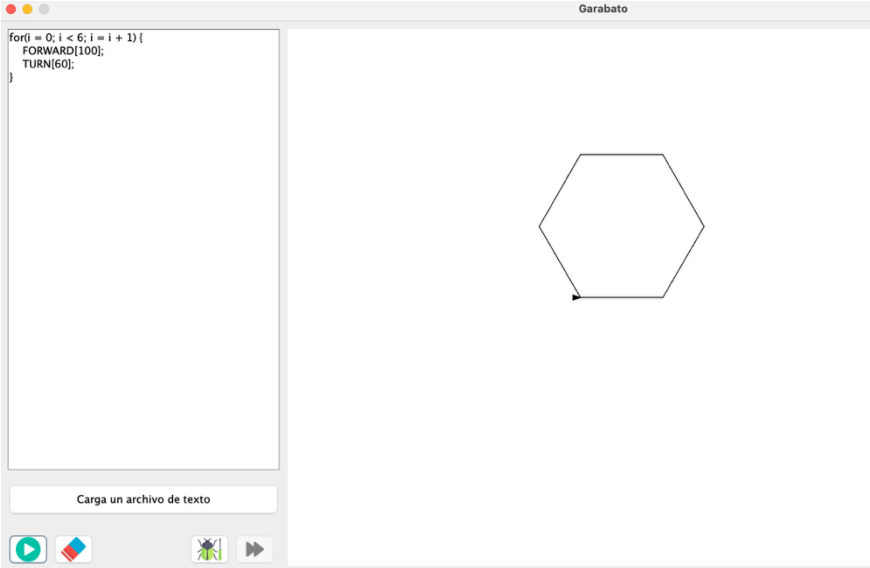
Intrucciones de modo de uso

- 1) Para avanzar de posición escriba FORWARD[X];
Donde X es lo que avanzara el cursor.
- 2) Para girar de dirección escriba TURN[X];
Donde X son los grados que girara el cursor.
- 3) Para cambiar el color con el que dibuja escriba COLOR[255,0,0]; usando valores RGB.
- 4) Se pueden crear funciones y estructuras de control como condicionales y ciclos.
- 5) Para empezar a usar el debugger presione el tercer boton a la derecha de la ultima linea, el código se ira ejecutando linea por linea mientras vaya presionando el botón de forward.

El mismo cuadrado pero cada lado en un color diferente. Se usa PenUP para reposicionar la tortuga antes de empezar.

```
PenUP[];  
FORWARD[100];  
TURN[90];  
PenDOWN[];  
COLOR[255,0,0]; FORWARD[100]; TURN[90];  
COLOR[0,255,0]; FORWARD[100]; TURN[90];  
COLOR[0,0,255]; FORWARD[100]; TURN[90];  
COLOR[255,165,0]; FORWARD[100]; TURN[90];
```

Polígonos regulares



Instrucciones de modo de uso

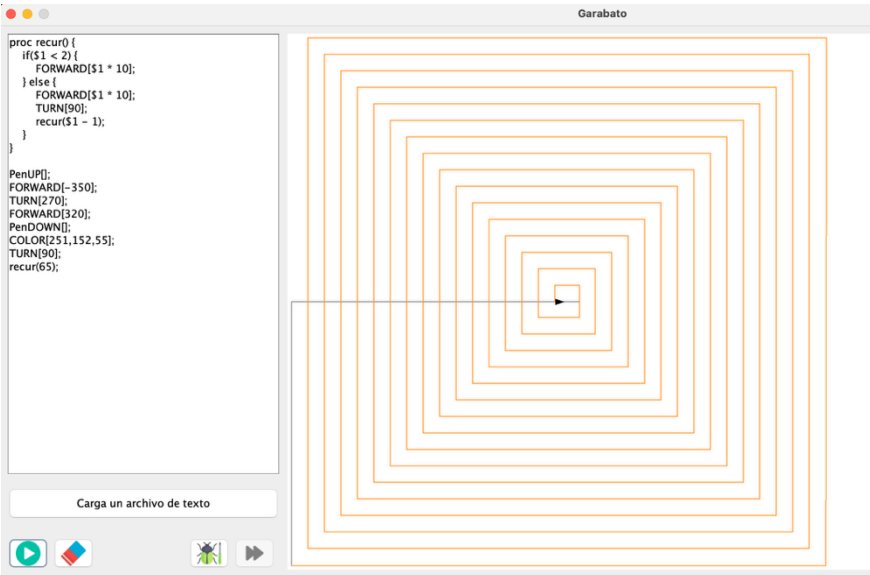
- 1) Para avanzar de posición escriba FORWARD[X];
Donde X es lo que avanzara el cursor.
- 2) Para girar de dirección escriba TURN[X];
Donde X son los grados que girara el cursor.
- 3) Para cambiar el color con el que dibuja escriba COLOR[255,0,0]; usando valores RGB.
- 4) Se pueden crear funciones y estructuras de control como condicionales y ciclos.
- 5) Para empezar a usar el debugger presione el tercer botón a la derecha de la última línea, el código se irá ejecutando línea por línea mientras vaya presionando el botón de forward.

Cualquier polígono regular de n lados se dibuja repitiendo n veces: FORWARD + TURN($360/n$). Por ejemplo, para un hexágono ($n=6$, ángulo= 60°):

```
for(i = 0; i < 6; i = i + 1) {  
    FORWARD[100];  
    TURN[60];  
}
```

Cambiando el 6 por 3 (triángulo, 120°), 5 (pentágono, 72°), 8 (octógono, 45°) o 10 (decágono, 36°) obtienes diferentes figuras.

Espiral cuadrada



Instrucciones de modo de uso

- 1) Para avanzar de posición escriba FORWARD[X];
Donde X es lo que avanzara el cursor.
- 2) Para girar de dirección escriba TURN[X];
Donde X son los grados que girara el cursor.
- 3) Para cambiar el color con el que dibuja escriba COLOR[255,0,0]; usando valores RGB.
- 4) Se pueden crear funciones y estructuras de control como condicionales y ciclos.
- 5) Para empezar a usar el debugger presione el tercer botón a la derecha de la última línea, el código se irá ejecutando línea por línea mientras vaya presionando el botón de forward.

Usando recursión con un procedimiento se puede construir una espiral. Cada llamada avanza un poco más que la anterior.

```

proc recur() {
  if($1 < 2) {
    FORWARD[$1 * 10];
  } else {
    FORWARD[$1 * 10];
    TURN[90];
    recur($1 - 1);
  }
}

PenUP[];
FORWARD[-350];
TURN[270];
FORWARD[320];
PenDOWN[];
COLOR[251,152,55];
TURN[90];
recur(65);

```

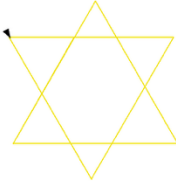
Estrella de David

```

func estrellaDavid() {
  COLOR[235,222,0];
  for(aux = 0; aux < 3; aux = aux + 1) {
    FORWARD[$1];
    TURN[120];
  }
  TURN[60];
  FORWARD[$1 - 50];
  TURN[120];
  FORWARD[$1 - 120];
  TURN[120];
  for(aux = 0; aux < 3; aux = aux + 1) {
    FORWARD[$1];
    TURN[120];
  }
}

estrellaDavid(200);

```



Instrucciones de modo de uso

- 1) Para avanzar de posición escriba FORWARD[X]; Donde X es lo que avanzara el cursor.
- 2) Para girar de dirección escriba TURN[X]; Donde X son los grados que girara el curso r.
- 3) Para cambiar el color con el que dibuja escriba COLOR[255,0,0]; usando valores RGB.
- 4) Se pueden crear funciones y estructuras de control como condicionales y ciclos.
- 5) Para empezar a usar el debugger presione el tercer boton a la derecha de la ultima linea, el código se ira ejecutando linea por linea mientras vaya presionando el botón de forward.






Define una función que dibuja la figura y luego llámala con el tamaño deseado.

```

func estrellaDavid() {
  COLOR[235,222,0];
  for(aux = 0; aux < 3; aux = aux + 1) {
    FORWARD[$1];
    TURN[120];
  }
  TURN[60];
  FORWARD[$1 - 50];
  TURN[120];
  FORWARD[$1 - 120];
  TURN[120];
  for(aux = 0; aux < 3; aux = aux + 1) {
    FORWARD[$1];
    TURN[120];
  }
}

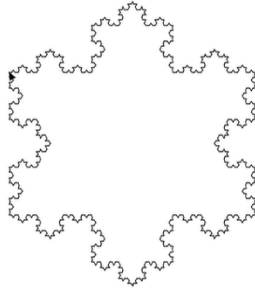
estrellaDavid(200);

```

Curva de Koch (copo de nieve fractal)

```
proc koch0 {
  if($1 == 0) {
    FORWARD[$2];
  } else {
    koch($1-1, $2/3);
    TURN[60];
    koch($1-1, $2/3);
    TURN[-120];
    koch($1-1, $2/3);
    TURN[60];
    koch($1-1, $2/3);
  }
}

koch(4, 300); TURN[-120];
koch(4, 300); TURN[-120];
koch(4, 300);
```



Intrucciones de modo de uso

- 1) Para avanzar de posición escriba FORWARD[X];
Donde X es lo que avanzara el cursor.
- 2) Para girar de dirección escriba TURN[X];
Donde X son los grados que girara el curso r.
- 3) Para cambiar el color con el que dibuja es criba COLOR[255,0,0]; usando valores RGB.
- 4) Se pueden crear funciones y estructuras de control como condicionales y ciclos.
- 5) Para empezar a usar el debugger presion e el tercer boton a la derecha de la ultima linea, el código se ira ejecutando linea por linea mientras presionando el botón de forward.

Un fractal recursivo clásico. El parámetro \$1 es el nivel de profundidad y \$2 es el tamaño del segmento inicial.

```
proc koch() {
  if($1 == 0) {
    FORWARD[$2];
  } else {
    koch($1-1, $2/3);
    TURN[60];
    koch($1-1, $2/3);
    TURN[-120];
    koch($1-1, $2/3);
    TURN[60];
    koch($1-1, $2/3);
  }
}

koch(4, 300); TURN[-120];
koch(4, 300); TURN[-120];
koch(4, 300);
```

Los fractales con nivel 4 o 5 pueden tardar unos segundos en dibujarse porque generan miles de instrucciones de forma recursiva. Es normal.

Árbol fractal

```
proc arbol0 {
  if($1 > 5) {
    FORWARD[$1];
    TURN[20];
    arbol($1 - 5);
    TURN[320];
    arbol($1 - 5);
    TURN[20];
    FORWARD[(-1) * ($1)];
  }
}

TURN[-90];
FORWARD[200];
COLOR[0, 143, 57];
TURN[180];
arbol(65);
```

Intrucciones de modo de uso

- 1) Para avanzar de posición escriba FORWARD[X];
Donde X es lo que avanzara el cursor.
- 2) Para girar de dirección escriba TURN[X];
Donde X son los grados que girara el cursor.
- 3) Para cambiar el color con el que dibuja escriba COLOR[255,0,0]; usando valores RGB.
- 4) Se pueden crear funciones y estructuras de control como condicionales y ciclos.
- 5) Para empezar a usar el debugger presione el tercer boton a la derecha de la ultima linea, el código se ira ejecutando linea por linea mientras vaya presionando el botón de forward.

Simula un árbol con ramas recursivas. Cada rama se divide en dos sub-ramas más cortas.

```
proc arbol() {
  if($1 > 5) {
    FORWARD[$1];
    TURN[20];
    arbol($1 - 5);
    TURN[320];
    arbol($1 - 5);
    TURN[20];
    FORWARD[(-1) * ($1)];
  }
}

TURN[-90];
FORWARD[200];
COLOR[0, 143, 57];
TURN[180];
arbol(65);
```

Espirográfo

```
func circulo0 {
  for(i = 0; i < 360; i = i + 1) {
    FORWARD[2];
    TURN[1];
  }
}

func espirografo {
  for(j = 0; j < 45; j = j + 1) {
    COLOR[j*30, j*40, j*20];
    circulo0;
    TURN[90];
  }
}

espirografo;
```

Intrucciones de modo de uso

- 1) Para avanzar de posición escriba FORWARD[X];
Donde X es lo que avanzara el cursor.
- 2) Para girar de dirección escriba TURN[X];
Donde X son los grados que girara el cursor.
- 3) Para cambiar el color con el que dibuja escriba COLOR[255,0,0]; usando valores RGB.
- 4) Se pueden crear funciones y estructuras de control como condicionales y ciclos.
- 5) Para empezar a usar el debugger presione el tercer boton a la derecha de la ultima linea, el código se ira ejecutando linea por linea mientras vaya presionando el botón de forward.

Combina círculos con rotaciones para generar patrones de espirografo con colores cambiantes.

```
func circulo() {
    for(i = 0; i < 366; i = i + 1) {
        FORWARD[2];
        TURN[1];
    }
}

func espirografo() {
    for(j = 0; j < 45; j = j + 1) {
        COLOR[j*30, j*40, j*20];
        circulo();
        TURN[90];
    }
}

espirografo();
```

Errores Comunes

Garabato no tiene mensajes de error detallados: si el código tiene un error de sintaxis simplemente no ejecuta nada. Estos son los errores más frecuentes y cómo evitarlos:

Error	Cómo evitarlo
Olvidar el punto y coma al final	Cada instrucción debe terminar con ; — incluyendo las que están dentro de bloques.
Usar minúsculas en los comandos	FORWARD, TURN, COLOR, PenUP y PenDOWN deben escribirse tal cual, con la combinación exacta de mayúsculas y minúsculas.
Olvidar los corchetes en los comandos	Los comandos Logo usan corchetes, no paréntesis. Correcto: FORWARD[100]; Incorrecto: FORWARD(100);
Incrementar el for con ++	Garabato no soporta i++. El incremento debe escribirse explícitamente: i = i + 1
Olvidar declarar variables antes de usarlas	Si usas una variable en una condición o expresión, asegúrate de haberle asignado un valor antes de ese punto.
Confundir proc y func	proc no regresa ningún valor. func sí, y debe incluir return. Usar func sin return puede causar comportamiento inesperado.

Referencia Rápida de Comandos

Comando	Qué hace
FORWARD[n] ;	Avanza n píxeles en la dirección actual. Negativo = retrocede.
TURN[n] ;	Gira n grados en sentido horario. Negativo = antihorario.
COLOR[R,G,B] ;	Cambia el color del trazo. Valores entre 0 y 255.
PenUP[] ;	Levanta el lápiz. La tortuga se mueve sin dibujar.
PenDOWN[] ;	Baja el lápiz. La tortuga vuelve a dibujar al moverse.
for(i=0; i<n; i=i+1){ }	Ciclo que se repite n veces.
while(condicion){ }	Ciclo que repite mientras la condición sea verdadera.
if(cond){ }else{ }	Ejecuta uno de dos bloques según la condición.
proc nombre(){ }	Declara un procedimiento sin valor de retorno.
func nombre(){ return v; }	Declara una función con valor de retorno.